



文章目录

优雅地开发 MCP 服务器

前言

(一) MCP 中的

MCP Server 中的基本概念

Resources, Prompts 和 T...

openai 协议

tool_calls 字段

Resources, Prompts,

Tools 和基本调试方法

快速开始



本文字数 2789 - 阅读时间约为 19 分钟 ☑ Mar 25, 2025

安装基本环境

使用 Inspector 进行调试

结余

挖坑





文章目录

前言

MCP Server 中的基本概念

Resources, Prompts 和 T...

openai 协议

tool_calls 字段

快速开始

安装基本环境

使用 Inspector 进行调试

结余

挖坑



H2 前言

90%



文章目录



在正式开始教大家如何开发自己的 MCP 服务器之前，我想，或前言 许有必要讲清楚几个基本概念。

MCP Server 中的基本概念

Resources, Prompts 和 T...

openai 协议

H2 MCP Server 中的基本概念

tool_calls 字段

快速开始

H3 Resources, Prompts 和 Tools

安装基本环境

在 [MCP 客户端协议](#) 中，讲到了 MCP 协议中三个非常重要的能力类别：使用 Inspector 进行调试

结余

挖坑

- **Resources**：定制化地请求和访问本地的资源，可以是文件系统、数据库、当前代码编辑器中的文件等等原本网页端的app 无法访问到的 **静态资源**。额外的 resources 会丰富发送给大模型的上下文，使得 AI 给我们更加精准的回答。
- **Prompts**：定制化一些场景下可供 AI 进行采纳的 prompt，比如如果需要 AI 定制化地返回某些格式化内容时，可以提供自定义的 prompts。
- **Tools**：可供 AI 使用的工具，它必须是一个函数，比如预定酒店、打开网页、关闭台灯这些封装好的函数就可以是一个 tool，大模型会通过 function calling 的方式来使用这些 tools。Tools 将会允许 AI 直接操作我们的电脑，甚至和现实世界发生交互。

90%

各位拥有前后端开发经验的朋友们，可以将 Resources 看成是「额外给予大模型的只读权限」，把 Tools 看成是「额外给予大模



MCP 客户端（比如 Claude Desktop, Sire 等）已经实现好了上述的前端部分逻辑。而具体提供什么资源，具体提供什么工具，则需要各位玩家充分想象了，也就是我们需要开发丰富多彩的

文章目录 MCP Server 来允许大模型做出更多有意思的工作。

前言 不过需要说明的一点是，目前几乎所有大模型采用了 openai 协议作为我们访问大模型的接入点。什么叫 openai 协议呢？

MCP Server 中的基本概念

H3 openai 协议

Resources, Prompts 和 T...

openai 协议 当我们使用 python 或者 typescript 开发 app 时，往往会安装一个名为 openai 的库，里面填入你需要使用的模型厂商、模型的基础 url、使用的模型类别来直接访问大模型。而各个大模型提供商也必须支持这个库，这套协议。

使用 Inspector 进行调试

比如我们在 python 中访问 deepseek 的服务就可以这么做：

结余

挖坑

```
from openai import OpenAI

client = OpenAI(api_key="<DeepSeek API Key>",
base_url="https://api.deepseek.com")

response = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "system", "content": "You are a helpful assistant"},
        {"role": "user", "content": "Hello"}
    ],
    stream=False
)
```



如果你点进这个 create 函数去看，你会发现 openai 协议需要大模型厂家支持的 feature 是非常非常多的：

文章目录



前言

@overload

def create(

MCP Server 中的基本概念

set,

Resources, Prompts 和 T...

messages:

openai 协议

Iterable[ChatCompletionMessageParam],

tool_calls 字段

model: Union[str, ChatModel],

audio: Optional[ChatCompletionAudioParam] |

快速开始

NotGiven = NOT_GIVEN,

安装基本环境

frequency_penalty: Optional[float] | NotGiven

= NOT_GIVEN,

使用 Inspector 进行调试

function_call:

结余

completion_create_params.FunctionCall | NotGiven =

NOT_GIVEN,

挖坑

functions:

Iterable[completion_create_params.Function] |

NotGiven = NOT_GIVEN,

logit_bias: Optional[Dict[str, int]] |

NotGiven = NOT_GIVEN,

logprobs: Optional[bool] | NotGiven =

NOT_GIVEN,

max_completion_tokens: Optional[int] |

NotGiven = NOT_GIVEN,

max_tokens: Optional[int] | NotGiven =

NOT_GIVEN,

metadata: Optional[Metadata] | NotGiven =

NOT_GIVEN,

modalities: Optional[List[Literal["text",

"audio"]]] | NotGiven = NOT_GIVEN,

n: Optional[int] | NotGiven = NOT_GIVEN,





文章目录

前言

MCP Server 中的基本概念

Resources, Prompts 和 Tools

openai 协议

tool_calls 字段

快速开始

安装基本环境

使用 Inspector 进行调试

结余

挖坑

```
Optional[ChatCompletionPredictionContentParam] |
NotGiven = NOT_GIVEN,
    presence_penalty: Optional[float] | NotGiven
= NOT_GIVEN,
    reasoning_effort: Optional[ReasoningEffort] |
NotGiven = NOT_GIVEN,
    response_format:
completion_create_params.ResponseFormat | NotGiven =
NOT_GIVEN,
    seed: Optional[int] | NotGiven = NOT_GIVEN,
    service_tier: Optional[Literal["auto",
"default"]] | NotGiven = NOT_GIVEN,
    stop: Union[Optional[str], List[str], None] |
NotGiven = NOT_GIVEN,
    store: Optional[bool] | NotGiven = NOT_GIVEN,
    stream: Optional[Literal[False]] | NotGiven =
NOT_GIVEN,
    stream_options:
Optional[ChatCompletionStreamOptionsParam] | NotGiven
= NOT_GIVEN,
    temperature: Optional[float] | NotGiven =
NOT_GIVEN,
    tool_choice:
ChatCompletionToolChoiceOptionParam | NotGiven =
NOT_GIVEN,
    tools: Iterable[ChatCompletionToolParam] |
NotGiven = NOT_GIVEN,
    top_logprobs: Optional[int] | NotGiven =
NOT_GIVEN,
    top_p: Optional[float] | NotGiven =
NOT_GIVEN,
    user: str | NotGiven = NOT_GIVEN,
    web_search_options:
completion_create_params.WebSearchOptions | NotGiven
= NOT_GIVEN,
    # Use the following arguments if you need to
```



文章目录

前言

MCP Server 中的基本概念

Resources, Prompts 和 Tools

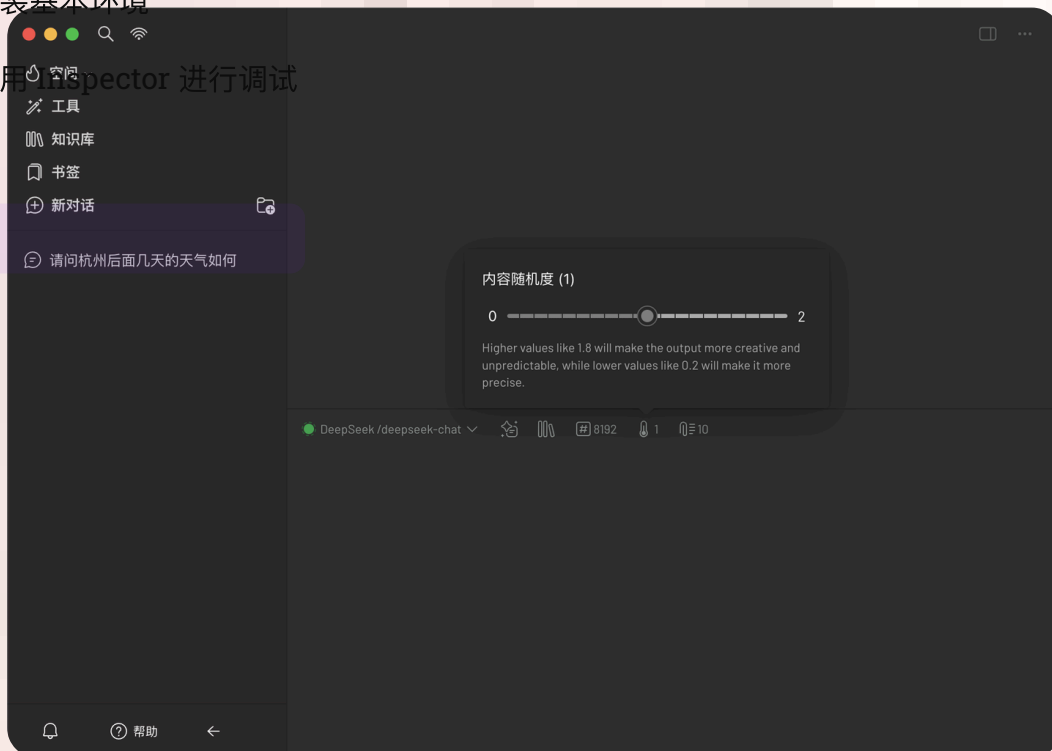
从上面的签名中，你应该可以看到几个很熟悉的参数，比如 `temperature`，`top_p`，很多的大模型使用软件中，有的会给你暴露这个参数进行调节。比如在 Sire 中，内容随机度就是 `temperature` 这个参数的图形化显示。

安装基本环境

使用 Inspector 进行调试

结语

挖坑



其实如你所见，一次普普通通调用涉及到的可调控参数是非常之多的。而在所有参数中，你可以注意到一个参数叫做 `tools`：



文章目录

前言

MCP Server 中的基本概念

快速开始

H3

tool_calls 字段

安装基本环境

使用上面的openai协议中，有一个名为 tools 的参数。tools 就是

结余 要求大模型厂商必须支持 function calling 这个特性，也就是我

挖坑 们提供一部分工具的描述（和 MCP 协议完全兼容的），在 tools

不为空的情况下，chat 函数返回的值中会包含一个特殊的字段

tool_calls ，我们可以运行下面的我写好的让大模型调用可

以查询天气的代码：

```
from openai import OpenAI

client = OpenAI(
    api_key="Deepseek API",
    base_url="https://api.deepseek.com"
)

# 定义 tools（函数/工具列表）
tools = [
```




文章目录

前言

MCP Server 中的基本概念

Resources, Prompts 和 T...

openai 协议

tool_calls 字段

快速开始

安装基本环境

使用 Inspector 进行调试

结余

挖坑

```
{
  "type": "function",
  "function": {
    "name": "get_current_weather",
    "description": "获取给定地点的天气",
    "parameters": {
      "type": "object",
      "properties": {
        "location": {
          "type": "string",
          "description": "城市, 比如杭州, 北京, 上海",
          "required": ["location"],
        }
      },
      "required": ["location"],
    }
  }
},
},
},
}

response = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "system", "content": "你是一个很有用的 AI"},
        {"role": "user", "content": "今天杭州的天气是什么?"},
    ],
    tools=tools, # 传入 tools 参数
    tool_choice="auto", # 可选: 控制是否强制调用某个工具
    stream=False,
)

print(response.choices[0].message)
```

运行上述代码，它的返回如下：



文章目录

前言

MCP Server 中的基本概念

Resources, Prompts 和 Tools

openai 协议

tool_calls 字段

快速开始

安装基本环境

使用 Inspector 进行调试

结语

挖坑

可以看到上面的 `tool_calls` 给出了大模型想要如何去使用我们给出的工具。需要说明的一点是，收到上下文的限制，目前一个问题能够让大模型调取的工具有限一般不超过 100 个，这个和大模型厂商的上下文大小有关系。奥，对了，友情提示，当你使用 MCP 客户端在使用大模型解决问题时，同一时间激活的 MCP Server 越多，消耗的 token 越多哦 :D

而目前 openai 的协议中，tools 是只支持函数类的调用。而函数类的调用往往是可以模拟出 Resources 的效果的。比如取资源，你可以描述为一个 tool。因此在正常情况下，如果大家要开发 MCP Server，最好只开发 Tools，另外两个 feature 还暂时没有得到广泛支持。



H2

快速开始

接下来就要让我们大显身手了！MCP 官方提供了几个封装好的

mcp sdk 来让我们快速开发一个 MCP 服务器。我看了一下，目

📖 文章目录

前使用最爽，最简单的是 python 的 sdk，所以我就用 python

前言 来简单演示一下了。

MCP Server 中的基本概念使用 typescript 开发也是没问题的，typescript

当然，如果想要使用 python 开发也是没问题的，python 的优势就是打包和部署更加简单。可以看我自用的一个模板库：

[mcp-server-template](#)

tool_calls 字段

H3

安装基本环境

快速开始

首先让我们打开一个项目，安装基本的库：

安装基本环境

使用 Inspector 进行调试

结余

```
pip install mcp "mcp[cli]" uv
```

挖坑

创建 server.py

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP('锦恢的 MCP Server', version="11.45.14")

@mcp.tool('add', '对两个数字进行实数域的加法')
def add(a: int, b: int) -> int:
    return a + b

@mcp.resource("greeting://{name}", 'greeting', '用于演示的一个资源协议')
def get_greeting(name: str) -> str:
```



```
@mcp.prompt('translate', '进行翻译的prompt')
def translate(message: str) -> str:
    return f'请将下面的话语翻译成中文: \n\n{message}'
```

文章目录



前言 上面的代码在装饰器的作用下非常容易读懂, @mcp.tool ,

MCP Server 中的基本概念 @mcp.resource 和 @mcp.prompt 分别实现了

Resource, Prompt 和 Tool

• 一个 greeting 协议的 resource

• 一个名为 translate 的 prompt
tool_calls 字段

快速开始

不明白装饰器是什么小白可以看看我之前的文章 [Python 进](#)

安装基本环境

[阶笔记 \(一\) 装饰器实现函数/类的注册](#)

使用 Inspector 进行调试

虽然注册器里面的第二个参数 description 不是必要的, 但是仍然建议写一下, 要不然大模型怎么知道你这个工具是干啥的。

结余

挖坑

H3 使用 Inspector 进行调试

我们可以使用 MCP 官方提供的 Inspector 工具对上面的 server 进行调试:



```
mcp dev server.py
```

90%

这会启动一个前端服务器并, 打开 <http://localhost:5173/> 后我们可以看到 inspector 的调试界面, 先点击左侧的



汇尘轩

web 建立通信

首页

博客

音乐

工具箱

其他

联系

Connect

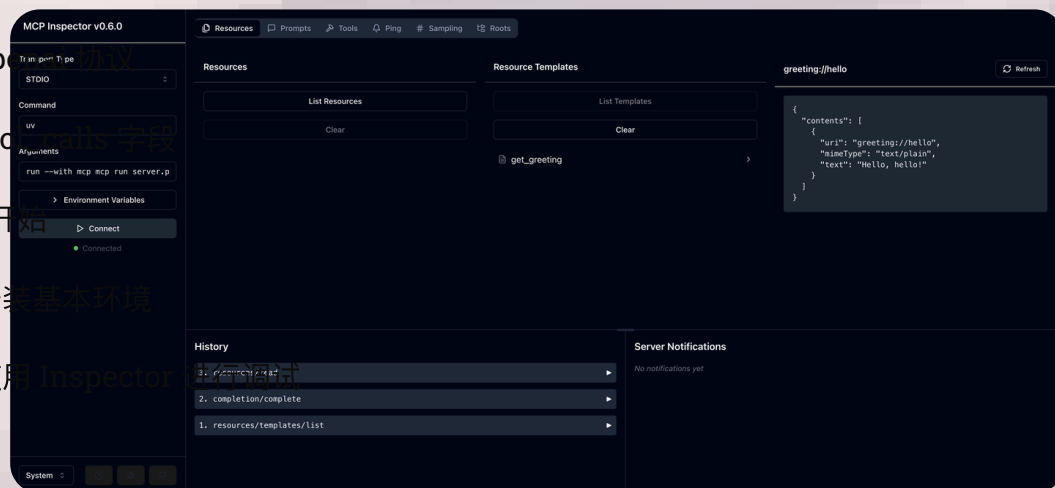
来运行我们的 server 并通过 stdio 为通信管道和

Fine, 可以开始愉快地进行调试了, Inspector 主要给了我们三个板块, 分别对应 Resources, Prompts 和 Tools。

文章目录

先看 Resources, 点击「Resource Templates」可以罗列所有注册的 Resource, 比如我们上文定义的 `get_greeting`, 你可以通过输入参数运行来查看这个函数是否正常工作。(因为一般 MCP Server 中的基本概念情况下的这个资源协议是会访问远程数据库或者微服务的)

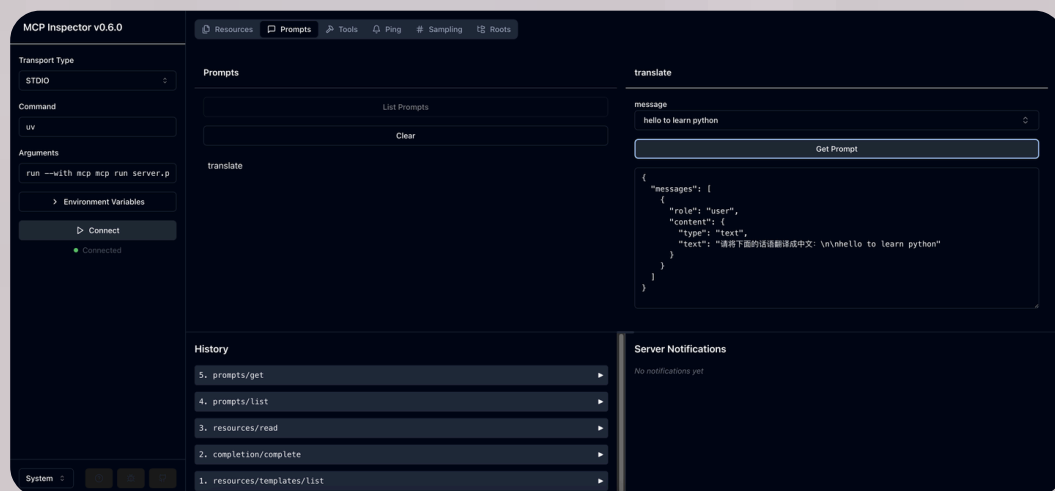
Resources, Prompts 和 T...



结语

挖坑

Prompts 端就比较简单了, 直接输入预定义参数就能获取正常的返回结果。

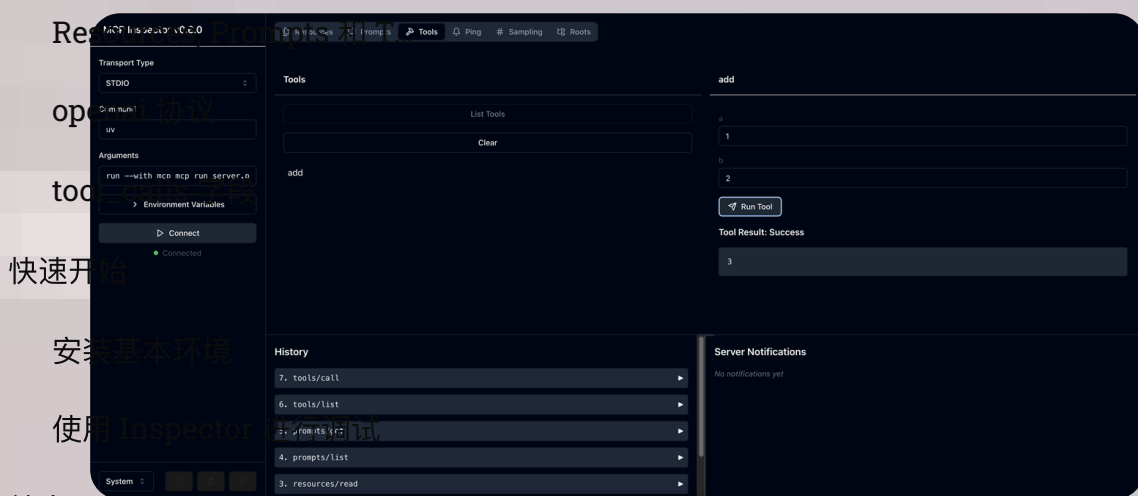


90%



openai 协议和各大 MCP 客户端广泛支持，因此，我们主要的服务端业务都应该是在写 tools。

我们此处提供的 tool 是实现一个简单的加法，它非常简单，我们输入 1 和 2 就可以直接看到结果是 3。我们后续会开发一个可以访问天气预报的 tool，那么到时候就非常需要一个这样的窗口来调试我们的天气信息获取是否正常了。



结论

挖坑

H2 结余

这篇文章，我们简单了解了 MCP 内部的一些基本概念，我认为这些概念对于诸位开发一个 MCP 服务器是大有裨益的，所以我认为有必要先讲一讲。

下面的文章中，我将带领大家探索 MCP 的奇境，一个属于 AI Agent 的时代快要到来了。



从上面的例子中，大家也能看出其实现在调试 MCP Server 的工具还不算齐全，所以我打算最近快速开发一款 vscode 插件，集合 Inspector 的所有功能和基础的大模型测试为一体。如果你开

文章目录基本的网络和磁盘访问的 MCP Server，这个调试工具也可以当成一个 Manus 客户端进行把玩。

前言

请大家期待吧！

MCP Server 中的基本概念

Resources, Prompts 和 T...

openai 协议

MCP

开发

agent

tool_calls 字段

快速开始

安装基本环境

使用 Inspector 进行调试

结余

挖坑

评论 (0)

0 / 200

支持markdown语法，latex数学公式，部分语言的语法高亮

预览



文章目录

前言

MCP Server 中的基本概念

Resources, Prompts 和 T...

openai 协议

暂时无人评价，是否要抢个沙发呢？

tool_calls 字段

快速开始

推荐文章

安装基本环境

使用 Inspector 进行调试

结余

挖坑



K Manus ...

Mar 09, 2025

22:43



K 使用 Hus...

Mar 21, 2025

15:18



K 通过 SSH...

Mar 19, 2025

19:38



汇尘轩



首页



博客



音乐



工具箱



其他



联系



文章目录



汇尘轩 - 锦恢的私人博客

前言

MCP Server 中的基本概念

Email: 1193466151@qq.com
Resources, Prompts 和 T...

Copyright © 2022-2025 锦恢 All Rights Reserved
openai 协议

苏ICP备2022010563号-2
tool_calls 字段

快速开始

安装基本环境

使用 Inspector 进行调试

结余

挖坑

相关链接



[Not Connected 官网](#)



[知乎@锦恢](#)



[Github@LSTM-Kirigaya](#)



[友情链接](#)



[Steam@锦恢](#)

90%